Gazebo Performance Profiling Report

Carlos Agüero

caguero@honurobotics.com

April 2026

Contents

1 Introduction **2**
1.1 Goals . . . 2
1.2 Approach . . . 2

2 Executive Summary **2**

3 Methodology and Reproduction Procedure **3**
3.1 Profiling Techniques . . . 3
3.2 Benchmark World Selection . . . 4
3.3 Hardware . . . 4
3.4 Software Versions . . . 4
3.5 Build Command . . . 4
3.6 Environment Setup . . . 4
3.7 Modified World SDFs . . . 4
3.8 Runtime Capture Command (per world) . . . 5
3.9 Loading Capture Command (per world) . . . 5
3.10 Data Organization . . . 5
3.11 Online Repository . . . 6

4 Loading-Time Analysis **6**
4.1 Loading Flamegraphs . . . 6
4.2 Key Loading-Time Findings . . . 7

5 Runtime Analysis Per World **9**
5.1 Runtime Flamegraphs . . . 9
5.2 3k_shapes_static (3000 static entities, headless, RTF=0) . . . 9
5.3 3k_shapes_dynamic (3000 dynamic entities, headless, RTF=0) . . . 10
5.4 sensors_nonrendering (IMU, mag, etc., headless, RTF=0) . . . 11
5.5 jetty_headless (complex world, headless, RTF=0) . . . 11
5.6 gpu_lidar (GPU lidar, headless-rendering, RTF=0) . . . 12
5.7 sensors_demo (6 rendering sensors, headless-rendering, RTF=0) . . . 12

6 Cross-Reference Analysis **13**
6.1 Gazebo Framework Overhead (visible in static/non-physics worlds) . . . 13
6.2 Physics Stack (dynamic worlds) . . . 14
6.3 Rendering Stack (sensor worlds) . . . 14

7 Optimization Recommendations (Priority-Ranked, Gazebo-Owned) **14**
7.1 Priority 1: Cache texture/image decoding in jetty-class worlds . . . 14
7.2 Priority 2: Reduce broadphase cost for worlds with many static bodies . . . 15
7.3 Priority 3: Optimize `BitmaskContactFilter::ignoresCollision` and `OdeCollisionObject::updateEngineData` . . . 16
7.4 Priority 4: Reduce loading time for large entity counts . . . 16
7.5 Priority 5: Investigate ECS View and thread barrier overhead . . . 16

8 Instrumentation Gaps **16**

9 Appendix **17**
9.1 A: Flamegraph SVG Locations . . . 17
9.2 B: GZ_PROFILE Instrumentation Added . . . 17
9.3 C: Loading Wall-Clock Times . . . 17
9.4 References . . . 17

1 Introduction

Gazebo is a widely used robotics simulator that integrates physics engines (DART, ODE), a 3D rendering pipeline (Ogre), sensor simulation, and an Entity Component System (ECS) into a single real-time loop. As simulation worlds grow in complexity — more models, more sensors, more detailed meshes — users frequently encounter real-time factor (RTF) degradation without clear visibility into what is consuming CPU time.

Prior profiling efforts in Gazebo have relied on the built-in Remotery profiler, which instruments explicitly annotated code scopes via `GZ_PROFILE` macros. While useful for understanding Gazebo's own subsystem timing, Remotery cannot see inside third-party libraries (DART, ODE, Ogre) or system-level costs (memory allocation, thread synchronization, dynamic linking). This leaves significant blind spots when diagnosing performance problems.

This report introduces **CPU flamegraph profiling** as a complementary technique. Flamegraphs, generated from Linux `perf` sampling data, capture the full call stack across all libraries — including DART's constraint solver, ODE's collision broadphase, Ogre's scene-graph traversal, and glibc's memory allocator. By combining flamegraphs with Remotery's structured scopes, we can attribute CPU time to specific subsystems and identify optimization targets with high confidence.

1.1 Goals

- Understand where CPU time goes across representative simulation workloads, with emphasis on Gazebo-owned code rather than third-party physics engines
- Measure startup/loading time and identify bottlenecks in world initialization
- Identify concrete, actionable optimization opportunities within the Gazebo codebase
- Establish a reproducible profiling methodology and benchmark suite for ongoing performance work

1.2 Approach

We selected 8 benchmark worlds covering distinct stress axes: entity count scaling (3000 shapes), collision-heavy physics, GPU lidar rendering, multi-sensor rendering, non-rendering sensors, and a realistic mixed-use scenario (jetty demo). Worlds were run at maximum speed (RTF=0) to saturate CPU and eliminate real-time pacing overhead. Both steady-state runtime and startup/loading captures were collected. A mix of static and dynamic worlds was used to separate Gazebo framework overhead from physics engine cost — the static variant of 3k_shapes (all models `<static>true</static>`) isolates ECS, transport, and scene management overhead by removing all DART/ODE work.

Additionally, 17 new `GZ_PROFILE` instrumentation points were added to `gz-physics/dartsim` to break the previously opaque `WorldForwardStep` call into observable sub-phases (collision detection, DART step, pose output, model construction).

---

2 Executive Summary

This report analyzes CPU distribution across 7 benchmark worlds (6 runtime + 4 loading captures) to identify where Gazebo spends time and where optimization investment should focus.

Top findings:

1. **3k_shapes loading takes 26 seconds** — dominated by `SceneBroadcasterPrivate::SceneGraphAddEntities` and SDF serialization (not parsing). The SceneBroadcaster serializes every entity's SDF back to string form on the first step. Loading time scales with entity count, not world complexity.
2. **Static worlds still exercise ODE broadphase** — `dxHashSpace::collide` at 2.6% even with 3000 static models. The collision system runs broadphase every step regardless of whether bodies move.
3. **Ogre2 scene-graph traversal dominates multi-sensor rendering** — `Ogre::Node::_getDerivedPosition` (8%) + `_getDerivedOrientationUpdated` (5.5%) in sensors_demo. Per-node transform updates are the hot path, not shader execution.

4. **Jetty loads JPEG textures at runtime** — `stbi__YCbCr_to_RGB_simd` (3.1%) + `stbi__jpeg_decode_bloc` (1.6%) during headless runtime, suggesting textures are decoded per-step or lazily on first use rather than cached.
5. **ECS View operations and thread barriers appear in framework overhead** — `BaseView::ResetNewEntityS` `Barrier::Wait`, and `pthread_cond_wait` are visible in the sensors and gpu_lidar worlds.

---

3 Methodology and Reproduction Procedure

3.1 Profiling Techniques

This study uses two complementary profiling techniques to achieve full-stack visibility into Gazebo's CPU usage.

CPU Flamegraphs are generated from Linux `perf` sampling data. The `perf` tool uses hardware performance counters to interrupt the running process at a fixed frequency (997 Hz in this study) and record the current call stack. These stack samples are then collapsed and rendered as an interactive SVG using Brendan Gregg's FlameGraph scripts [1]. In a flamegraph, the x-axis represents the proportion of total CPU samples (wider = more time), and the y-axis represents call stack depth (bottom = entry point, top = leaf function where time is actually spent). The resulting SVG is interactive: clicking any frame re-renders the chart with that frame as the new baseline, effectively zooming into that subtree. Flamegraphs capture *every* function in the call stack, including third-party libraries (DART, ODE, Ogre), system libraries (glibc malloc, pthreads), and kernel calls — making them ideal for identifying bottlenecks that span library boundaries.

Two types of flamegraph captures were performed:

- **Runtime captures**: `perf` attaches to a running Gazebo process and samples for 30 seconds during steady-state simulation. This reveals where CPU goes during normal operation.
- **Loading captures**: `perf` wraps the entire process launch (`perf record -- gz-sim-main --iterations 1 <world>`), capturing startup, SDF parsing, physics engine initialization, and scene construction. This reveals where loading time goes.

To produce readable flamegraphs, the code must be compiled with debug symbols (`-g` via `RelWithDebInfo`) so that `perf` can map instruction addresses to function names, and with frame pointers (`-fno-omit-frame-pointer`) so that `perf` can walk the call stack reliably. Without frame pointers, stack traces break and the flamegraph contains `[unknown]` frames. An alternative is `perf record --call-graph dwarf`, which uses DWARF debug info for stack unwinding but produces 3-5x larger capture files.

The sampling frequency of 997 Hz (rather than 1000 Hz) avoids lock-step aliasing with system timers that may fire at 1 kHz. The Linux `perf_event_paranoid` sysctl must be set to 1 or lower to allow user-mode CPU sampling.

GZ_PROFILE (Remotery Profiler) is Gazebo's built-in instrumentation system. Developers annotate code with `GZ_PROFILE("scope_name")` macros that record entry/exit timestamps and send them to a Remotery server. A browser-based visualizer connects via WebSocket and displays a real-time timeline of named scopes per thread. Unlike flamegraphs, Remotery shows only *explicitly annotated* scopes — it cannot see inside libraries that lack instrumentation. However, it provides structured, named timing data that is easy to interpret and correlate with specific Gazebo subsystems.

The `GZ_PROFILE` macros are compiled as no-ops unless `ENABLE_PROFILER=ON` is passed at CMake time, ensuring zero overhead in production builds. Four Gazebo libraries support this flag: gz-sim, gz-physics, gz-rendering, and gz-sensors.

In this study, `ENABLE_PROFILER=ON` was used for all captures. This means Remotery instrumentation is active during `perf` captures — the `GZ_PROFILE` calls appear as identifiable function frames in the flamegraph (e.g., `_rmt_BeginCPUSample` at ~1.5%), serving as named landmarks that help attribute flamegraph regions to specific Gazebo subsystems. The overhead is small and consistent across captures.

As part of this study, 17 new `GZ_PROFILE` points were added to `gz-physics/dartsim` to break the pre-

viously opaque `WorldForwardStep` call into observable sub-phases: pre-step gravity, DART world step, NaN checking, pose output, collision detection wrappers, kinematics queries, and SDF model construction methods.

3.2 Benchmark World Selection

Seven benchmark worlds were selected to cover distinct stress axes. All worlds were modified to run at maximum speed (`<real_time_factor>0</real_time_factor>`) to saturate CPU and eliminate the real-time pacing overhead from the flamegraphs. A mix of static and dynamic worlds separates Gazebo framework overhead from physics engine cost:

- **Static worlds** (3k_shapes_static, sensors, jetty paused): isolate ECS iteration, transport publishing, and rendering pipeline cost by removing all DART/ODE physics work.
- **Dynamic worlds** (3k_shapes, jetty headless): show the full picture including physics. Comparing dynamic 3k_shapes with its static variant reveals the pure physics engine cost.
- **Rendering/sensor worlds** (gpu_lidar, sensors_demo): exercise the Ogre rendering pipeline and sensor simulation. These require `gz topic -e` subscribers to force the sensor system to actually render and publish data — without subscribers, the pipeline skips rendering work entirely.

3.3 Hardware

- **CPU**: Intel Core Ultra 9 285HX, 24 cores (no HT), x86_64
- **GPU**: NVIDIA RTX PRO 3000 Blackwell Generation Laptop GPU
- **NVIDIA Driver**: 580.126.09 (used for headless EGL rendering via Ogre2)
- **Kernel**: 6.17.0-20-generic
- **GCC**: 13.3.0

3.4 Software Versions

- **gz-sim**: 11.0.0-pre1 (workspace source build, main branch)
- **gz-physics**: source build with 17 GZ_PROFILE points added to dartsim
- **DART**: system-packaged libdart 6.13.2 (no debug symbols)
- **perf**: 6.17.13
- **FlameGraph**: Brendan Gregg's scripts at `~/rotary_ws/tools/FlameGraph/`

3.5 Build Command

```
cd ~/rotary_ws
colcon build --merge-install \
  --cmake-args -DENABLE_PROFILER=ON \
               -DCMAKE_BUILD_TYPE=RelWithDebInfo \
               -DCMAKE_CXX_FLAGS="-fno-omit-frame-pointer" \
               -DCMAKE_C_FLAGS="-fno-omit-frame-pointer"
```

3.6 Environment Setup

```
source ~/rotary_ws/install/setup.bash
export GZ_CONFIG_PATH=~/rotary_ws/install/share/gz:$GZ_CONFIG_PATH
export GZ_SIM_RESOURCE_PATH=~/jetty_demo/jetty_demo/models:$GZ_SIM_RESOURCE_PATH
sudo sysctl kernel.perf_event_paranoid=1
```

3.7 Modified World SDFs

All benchmark worlds were copied to `~/rotary_ws/profiling/worlds/` with the SDF tag `real_time_factor` set to `0` to saturate CPU (eliminate pacing overhead). `3k_shapes_static.sdf` additionally has the `static` tag set to `true` on all 3004 models.

3.8 Runtime Capture Command (per world)

```
# Launch headless
~/rotary_ws/install/libexec/gz/sim/gz-sim-main -s -r <world.sdf> &
PID=$!; sleep 5

# For sensor worlds: gz topic -e -t <topic> > /dev/null &

# Record 30s at 997 Hz
perf record -F 997 --call-graph fp -p $PID -o perf.data sleep 30
kill $PID

# Generate flamegraph
perf script -i perf.data | stackcollapse-perf.pl | flamegraph.pl > flamegraph.svg
```

3.9 Loading Capture Command (per world)

```
perf record -F 997 --call-graph fp -o perf_loading.data \
  -- gz-sim-main -s -r --iterations 1 <world.sdf>
```

3.10 Data Organization

All profiling data is stored under `~/rotary_ws/profiling/` with the following structure:

```
profiling/
  worlds/                          # Modified benchmark world SDFs
    3k_shapes.sdf                  #   Dynamic variant (RTF=0)
    3k_shapes_static.sdf           #   Static variant (RTF=0, all models static)
    sensors.sdf                    #   Non-rendering sensors (RTF=0)
    jetty.sdf                      #   Complex real-world scene (RTF=0)
    gpu_lidar_sensor.sdf           #   GPU lidar (RTF=0)
    sensors_demo.sdf               #   Multi-sensor rendering (RTF=0)
  captures/
    flamegraphs/
      runtime/                     # Steady-state captures (30s each)
        perf_<label>.data          #   Raw perf binary data
        <label>.folded             #   Collapsed stacks (text)
        <label>.svg                #   Interactive flamegraph
      loading/                     # Startup captures (1 iteration)
        perf_<label>_loading.data  #   Raw perf binary data
        <label>_loading.folded     #   Collapsed stacks (text)
        <label>_loading.svg        #   Interactive flamegraph
        <label>_wallclock.txt      #   Wall-clock loading time
    remotery/                      # Remotery screenshots (manual)
  analysis/
    findings_report.tex            # This report (LaTeX source)
    findings_report.pdf            # This report (compiled)
    findings_report.md             # Original markdown source
    header.tex                     # LaTeX preamble/formatting
    figures/                       # Flamegraph PNGs for the report
  scripts/
    gz_flamegraph.sh               # Runtime capture script
    gz_loading_flamegraph.sh       # Loading capture script
    capture_all.sh                 # Orchestrates all captures
```

File types:

- `perf_*.data` — raw binary capture from `perf record`. Contains all stack samples. Can be re-analyzed with `perf report` or `perf script`. Large files; can be deleted once `.folded` files are generated.

- `*.folded` — collapsed stack traces in text format (one line per unique stack, with sample count). Input for `flamegraph.pl`. Also useful for CLI analysis with `awk/sort`.
- `*.svg` — interactive flamegraph. Open in a web browser; click frames to zoom, Ctrl+F to search.
- `*_wallclock.txt` — wall-clock time for loading captures, recorded via the `time` command.

3.11 Online Repository

All interactive flamegraphs and the PDF report are published on GitHub Pages at:

https://caguero.github.io/gz-profiling-flamegraphs/

The repository (https://github.com/caguero/gz-profiling-flamegraphs) is organized by date so that future benchmark runs are stored alongside historical data without overwriting. The structure is:

```
gz-profiling-flamegraphs/
  index.html                  # Landing page listing all runs
  2026-04-21/                 # This benchmark run
    index.html                # Per-run index with all flamegraph links
    findings_report.pdf       # This report
    runtime/                  # Interactive runtime flamegraph SVGs
    loading/                  # Interactive loading flamegraph SVGs
  YYYY-MM-DD/                 # Future runs
```

Flamegraph thumbnails in this report link to the corresponding interactive SVG on GitHub Pages. Click any thumbnail to open the full flamegraph in a browser with click-to-zoom, hover tooltips, and search functionality.

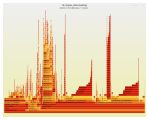
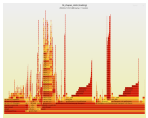
---

4 Loading-Time Analysis

World	Wall-clock	Samples	Key observation
3k_shapes_static	**26.0s**	25,962	Dominated by entity creation
3k_shapes_dynamic	**26.6s**	26,545	Nearly identical to static — physics init is cheap
jetty	**7.8s**	7,718	SdfModelSerializer::Serialize + plugin loading
sensors_nonrendering	2.4s	368	Fast — few entities
gpu_lidar	0.06s	5	Trivial — few entities, rendering initializes at runtime
sensors_demo	0.03s	5	Trivial — few entities, rendering initializes at runtime

4.1 Loading Flamegraphs

Click any thumbnail to open the full interactive flamegraph in a browser (supports click-to-zoom, hover tooltips, and Ctrl+F search).

World	Wall-clock	Flamegraph
3k_shapes_static	26.0s	
3k_shapes_dynamic	26.6s	
jetty	7.8s	
sensors_nonrendering	2.4s	*Fast — few entities, no significant loading cost*
gpu_lidar	0.06s	*Loads instantly — few entities*
sensors_demo	0.03s	*Loads instantly — few entities*

4.2 Key Loading-Time Findings

3k_shapes (26s)

Loading flamegraph shows `SceneBroadcasterPrivate:: SceneGraphAddEntities` as a top self-time leaf (3% of loading in static variant). The loading time is dominated by constructing 3000 ECS entities and DART skeletons. Static vs dynamic loading time is nearly identical (26.0 vs 26.6s), confirming that DART skeleton construction cost is per-entity, not per-dynamic-entity.

`SceneGraphAddEntities`
: (gz-sim, SceneBroadcaster.cc) Constructs a directed acyclic graph tracking parent-child relationships for new entities and serializes their pose and component data into protobuf messages for transport publication. Scales linearly with entity count.

`DownloadAssets`
: (gz-sim, ServerPrivate.cc) Despite no remote assets in any benchmark world, this function consumes a significant fraction of every world's loading time. The SDF file is parsed **twice** during startup by design:

World	% of loading	Estimated time
sensors_nonrendering	29.9%	0.7s
jetty	13.9%	1.0s
3k_shapes_static	9.1%	2.3s
3k_shapes_dynamic	8.3%	2.2s

1. **First parse** (Server.cc:93): `LoadSdfRootHelper` parses the full SDF quickly (1.3%, barely visible in the flamegraph). The server then **intentionally strips all models, actors, and lights** from the result (Server.cc:104–108, `ClearModels/ClearActors/ClearLights`) and creates simulation runners with an empty world shell. This allows the GUI to start immediately without waiting for asset downloads.
2. **Second parse** (ServerPrivate.cc:787, inside `DownloadAssets`): `LoadSdfRootHelper` is called **again** on a background thread, this time fully resolving all model references via `Root::Load → readFile → init → initDoc → initXml` (deeply recursive). After parsing, the complete world (now including any freshly downloaded Fuel models) is injected back into the runner via `SetWorldSdf` (line 840), and `SetCreateEntities` (line 844) triggers entity creation. This second parse is visible in the flamegraph of **every world** — not just 3k_shapes.

The architecture is intentional: parse once for fast GUI startup, parse again in the background

to get the full models. However, for worlds without Fuel URIs (all benchmark worlds tested), the first parse already has all the models — they are discarded and then re-parsed for no reason, costing 0.7–2.3 seconds per startup. The optimization: when no Fuel URIs are queued, skip the second parse and keep the models from the first parse instead of discarding them. Additionally, `FetchQueuedAssets` creates a local `WorkerPool(2)`, spawning 2 threads for an empty download queue — also skippable.

jetty (7.8s)

The bottleneck is `SdfModelSerializer::Serialize`, called from `SceneBroadcaster::PostUpdate → EntityComponentManager::AddEntityToMessage`. This serializer converts every model's SDF representation back to a string via recursive `Element::ToString → PrintValuesImpl` calls. The recursive depth (3-5 levels of `ToString`, 2-5 levels of `PrintValuesImpl`) amplifies the cost for complex models.

`SdfModelSerializer::Serialize`
: (gz-sim, EntityComponentManager.cc) Called only once — on the first simulation step when all entities are new. The call path is: `SceneBroadcaster::PostUpdate` checks `HasNewEntities()`, which triggers `ChangedState()` → iterates over all `newlyCreatedEntities` → calls `AddEntityToMessage()` for each → invokes `SdfModelSerializer::Serialize` which recursively converts the SDF to string via `Element::ToString → PrintValuesImpl`. On subsequent steps, `HasNewEntities()` is false so this code path is never reached again. The cost is a one-time per-loading spike, but for complex models with deeply nested SDF elements, it dominates the first step.

`do_lookup_x`
: (glibc dynamic linker, 4.8%) Resolves symbol references when shared library plugins are loaded. Each plugin triggers symbol relocation across all loaded libraries. Appears in top-10 for every world (2-9%).

`DownloadAssets`
: (gz-sim, ServerPrivate.cc, 13.9% in jetty) Same redundant second SDF parse as described under 3k_shapes above. In jetty, this costs ~1.0s of the 7.8s loading time — the SDF is re-parsed on a background thread despite no Fuel URIs being present.

Common observations

`do_lookup_x` (dynamic linker) appears in top-10 for every loading capture (2-9%).

`pybind11::initialize_interpreter`
: (gz-sim, Server.cc) The Python interpreter is initialized **unconditionally** at server construction (`Server::Server → pybind11::initialize_interpreter → Py_InitializeFromConfig`), even when no Python system plugins are referenced in the world file. None of the benchmark worlds use Python plugins, yet every loading capture pays a fixed ~200–260ms cost for interpreter initialization and module imports:

World	% of loading	Estimated time
sensors_nonrendering	7.5%	180ms
jetty	3.3%	257ms
3k_shapes_static	1.0%	260ms
3k_shapes_dynamic	0.9%	239ms

The fix: lazy-initialize the Python interpreter only when a `python_system_loader` plugin is actually requested by the world.

---

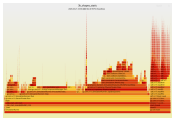
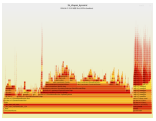
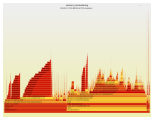
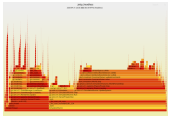
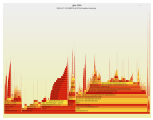
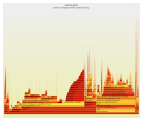
5 Runtime Analysis Per World

All runtime captures were recorded for 30 seconds at 997 Hz with the real-time factor target set to 0 (run as fast as possible). The measured RTF column shows the actual achieved performance.

World	Measured RTF	Samples	Key observation
3k_shapes_static	0.04	557K	ODE broadphase at 2.6% despite all models static
3k_shapes_dynamic	0.01	611K	10.2% unresolved DART + 4.5% getSkeleton
sensors_nonrendering	1.94	603K	Thread sync overhead 4% (Barrier::Wait, pthread)
jetty_headless	0.11	578K	Collision 21.9% + JPEG texture decode 6%
gpu_lidar	1.44	868K	NVIDIA EGL driver 6.9%, Ogre2 culling 3%
sensors_demo	1.34	691K	Ogre2 scene-graph traversal 22%

5.1 Runtime Flamegraphs

Click any thumbnail to open the full interactive flamegraph in a browser (supports click-to-zoom, hover tooltips, and Ctrl+F search).

World	Stress Axis	Flamegraph
3k_shapes_static	3000 static entities	
3k_shapes_dynamic	3000 dynamic entities	
sensors_nonrendering	IMU, mag, etc.	
jetty_headless	Complex real-world scene	
gpu_lidar	GPU lidar rendering	
sensors_demo	6 rendering sensors	

5.2 3k_shapes_static (3000 static entities, headless, RTF=0)

Purpose: Isolate Gazebo framework overhead with zero physics work.

Leaf function	%	Category
`void` (demangling artifact)	35.4	(noise)
`operator` (demangling)	12.3	(noise)
`unsigned` (demangling)	6.4	(noise)
`WorldForwardStep`	3.1	gz-physics (self-time in loop setup)

Leaf function	%	Category
`[[vdso]]` clock_gettime	3.0	timing
`dxHashSpace::collide`	**2.6**	ODE broadphase on static geometry
`DegreeOfFreedom::getPosition`	2.5	DART accessor
`Frame::getWorldTransform`	1.8	DART accessor
`CollisionGroup::updateSkeletonSource`	1.7	DART collision prep
`_rmt_BeginCPUSample`	1.5	Remotery profiler overhead

Key finding: Even with all models static, ODE broadphase collision runs every step at 2.6% of CPU. `WorldForwardStep` has 3.1% self-time — the per-link loop iterating over 3000 links. DART accessors (`getPosition`, `getWorldTransform`) are called per-entity per-step.

`WorldForwardStep`
(gz-physics, SimulationFeatures.cc) Main simulation entry point that orchestrates pre-step gravity, the DART world step, NaN checking, and pose output to the ECS. The 3.1% self-time is the per-link iteration over 3000 links.

`dxHashSpace::collide`
(ODE) Broad-phase collision detection using spatial hashing. Sweeps all shapes in the hash space to find potentially overlapping pairs — including static-vs-static pairs that can never produce contacts.

`DegreeOfFreedom::getPosition`
(DART) Returns the current generalized coordinate of a joint DOF. Called per-DOF per-step during skeleton state queries.

`Frame::getWorldTransform`
(DART) Computes world-space transform by traversing up the kinematic chain and accumulating parent transforms. Required for pose synchronization.

`_rmt_BeginCPUSample`
(Remotery) Records entry to a profiling scope. The 1.5% overhead is the cost of Remotery instrumentation with `ENABLE_PROFILER=ON`; disappears when disabled.

5.3 3k_shapes_dynamic (3000 dynamic entities, headless, RTF=0)

Purpose: Full physics with 3000 bodies. Compare with static variant.

Leaf function	%	Category
`[libdart.so.6.13.2]` unresolved	10.2	DART (stripped)
`BodyNode::getSkeleton`	4.5	DART accessor
`[[vdso]]`	3.0	timing
`BoxedLcpConstraintSolver`	2.1	DART LCP solver
`buildConstrainedGroups`	2.0	DART constraint setup
`_rmt_BeginCPUSample`	1.5	Remotery

Static→dynamic delta: LCP solver appears (2.1%), `buildConstrainedGroups` appears (2.0%), `BodyNode::getSkeleton` jumps from 0% to 4.5%. The unresolved `libdart` bucket grows from 0% to 10.2%. This delta (~20% of total) is the pure physics engine cost for 3000 bodies.

`BodyNode::getSkeleton`
(DART) Returns a pointer to the skeleton that owns a body node. A simple accessor, but called per-body in the inner solver loop for every contact constraint — hence 4.5% with 3000 dynamic bodies.

`BoxedLcpConstraintSolver`
(DART) Solves contact constraints using a Linear Complementarity Problem (LCP) formulation.

Determines contact forces to prevent interpenetration. Cost scales with the number of active contacts.

buildConstrainedGroups
(DART) Partitions contacts into constraint islands (connected components of interacting bodies) to reduce the LCP problem size. Each island is solved independently.

5.4 sensors_nonrendering (IMU, mag, etc., headless, RTF=0)

Leaf function	%	Category
`[unknown]`	34.6	kernel/stripped libs
`[libdart.so.6.13.2]`	5.4	DART
`[[vdso]]`	2.5	timing
`BodyNode::getSkeleton`	1.8	DART
`pthread_cond_wait`	**1.8**	thread sync
`_rmt_BeginCPUSample`	1.5	Remotery
`pthread_mutex_lock`	**1.1**	thread sync
`Barrier::Wait`	**1.0**	gz-sim thread barrier
`BaseView::ResetNewEntityState`	**0.8**	gz-sim ECS

Key finding: Thread synchronization (`pthread_cond_wait` + `pthread_mutex_lock` + `Barrier::Wait`) totals ~4%. ECS view operations (`BaseView::ResetNewEntityState`) visible. This is framework overhead — not physics, not rendering.

Barrier::Wait
(gz-sim, Barrier.hh) Thread synchronization barrier that blocks worker threads until all have reached the synchronization point, then releases them together. Used to coordinate PreUpdate/Update/PostUpdate system execution phases.

pthread_cond_wait
(glibc) Underlying POSIX condition variable wait used by `Barrier::Wait` and other synchronization primitives. The thread sleeps until signaled.

BaseView::ResetNewEntityState
(gz-sim, detail/BaseView.hh) Clears the "new entity" flag on all cached entities in an ECS view at the start of each step. Allows systems to distinguish newly created entities from existing ones. Cost scales with view size.

5.5 jetty_headless (complex world, headless, RTF=0)

Leaf function	%	Category
`dxHashSpace::collide`	**21.9**	ODE broadphase
`[unknown]`	10.7	kernel/stripped
`stbi__YCbCr_to_RGB_simd`	**3.1**	JPEG decode
`Frame::getWorldTransform`	1.8	DART
`stbi__jpeg_decode_block`	**1.6**	JPEG decode
`[libdart.so.6.13.2]`	1.5	DART
`BitmaskContactFilter::ignoresCollision`	**1.5**	gz-physics owned
`OdeCollisionObject::updateEngineData`	**1.2**	gz-physics owned
`stbi__convert_format`	1.2	image processing
`gz::common::Image::Width`	0.9	image property query

Key finding: JPEG texture decoding (`stbi_*` functions) consumes ~6% of CPU during headless runtime. The full call path reveals this happens inside `SceneManager::CreateVisual` — visuals are created lazily during simulation on the render thread, not at startup. Each new visual triggers

`Ogre2Material::SetTextureMapDataImpl` → `gz::common::Image::RGBAData` → `stbi_*` to decode all material textures from JPEG. `BitmaskContactFilter` and `OdeCollisionObject::updateEngineD` are Gazebo-owned at 1.5% and 1.2%.

`dxHashSpace::collide`
(ODE) Broad-phase collision using spatial hashing. At 21.9%, it dominates jetty due to the large number of collision shapes in the marina environment.

`stbi__YCbCr_to_RGB_simd`
(stb_image, in gz-common) SIMD-accelerated JPEG color-space conversion from YCbCr to RGB. Called during lazy texture decoding when `SceneManager::CreateVisual` loads material textures on the render thread.

`stbi__jpeg_decode_block`
(stb_image) Reconstructs an 8×8 JPEG image block via inverse DCT. Works in tandem with the color-space converter above.

`BitmaskContactFilter::ignoresCollision`
(gz-physics, EntityManagementFeatures.cc) Checks whether two collision shapes should be filtered based on user-defined bitmask values. Called once per broad-phase pair before narrow-phase testing.

`OdeCollisionObject::updateEngineData`
(DART/gz-physics) Synchronizes DART skeleton pose state to ODE collision object wrappers. Called per collision object per step, even when the object hasn't moved.

5.6 gpu_lidar (GPU lidar, headless-rendering, RTF=0)

Leaf function	%	Category
`[unknown]`	21.4	kernel/stripped
`[[vdso]]`	7.2	timing/GPU sync
`[libnvidia-eglcore]`	**6.9**	NVIDIA driver
`[libdart.so.6.13.2]`	6.4	DART
`BodyNode::getSkeleton`	4.5	DART
`BoxedLcpConstraintSolver`	4.1	DART solver
`Ogre::MovableObject::cullFrustum`	1.2	Ogre2 culling
`Ogre::SceneManager::cullFrustum`	1.0	Ogre2 culling
`BaseView::ResetNewEntityState`	**0.9**	gz-sim ECS
`Ogre::SceneManager::updateAllLodsThread`	0.8	Ogre2 LOD

Key finding: NVIDIA EGL driver at 6.9% — GPU driver overhead for headless rendering. Ogre2 frustum culling and LOD updates visible but small (~3% combined). Physics still dominates despite lidar being the target.

`libnvidia-eglcore`
(NVIDIA driver) GPU driver overhead for EGL-based headless rendering. Handles GPU command submission, buffer management, and synchronization without an X server.

`Ogre::MovableObject::cullFrustum`
(Ogre2) Tests whether a renderable object intersects the camera frustum. Called per-object per-camera to determine visibility.

`Ogre::SceneManager::updateAllLodsThread`
(Ogre2) Updates level-of-detail selection for all objects based on distance to camera. Runs on a worker thread.

5.7 sensors_demo (6 rendering sensors, headless-rendering, RTF=0)

Leaf function	%	Category
`[unknown]`	19.6	kernel/stripped
`Ogre::Node::_getDerivedPosition`	**8.0**	Ogre2 scene graph
`[libnvidia-eglcore]`	**5.6**	NVIDIA driver
`Ogre::Node::_getDerivedOrientationUpdated`	**5.5**	Ogre2 scene graph
`[libdart.so.6.13.2]`	4.6	DART
`[[vdso]]`	4.1	timing
`Ogre::Node::updateFromParentImpl`	**2.9**	Ogre2 scene graph
`Ogre::Frustum::updateWorldSpaceCornersImpl`	**2.6**	Ogre2
`Ogre::RenderSystem::_makeRsProjectionMatrix`	**2.4**	Ogre2
`Ogre::SceneManager::cullFrustum`	1.5	Ogre2

Key finding: Ogre2 scene-graph transform traversal totals ~22% (`_getDerivedPosition` + `_getDerivedOrientationUpdated` + `updateFromParentImpl` + frustum updates). This is the dominant cost for multi-sensor rendering — not shader execution, not GPU submission, but CPU-side scene-graph walking. NVIDIA driver at 5.6%.

`Ogre::Node::_getDerivedPosition`
(Ogre2) Computes world-space position of a scene node by traversing up the parent chain. Called per-node per-camera — with 6 cameras, each node is queried 6 times per frame.

`Ogre::Node::_getDerivedOrientationUpdated`
(Ogre2) Computes world-space orientation of a scene node, re-deriving from parent if the dirty flag is set. Companion to `_getDerivedPosition`.

`Ogre::Node::updateFromParentImpl`
(Ogre2) Propagates parent transform changes down to child nodes. Recursive tree traversal that updates local-to-world matrices when any ancestor moves.

`Ogre::Frustum::updateWorldSpaceCornersImpl`
(Ogre2) Computes the 8 frustum corner points in world-space for each camera. Used for visibility culling and shadow map computation.

`Ogre::ForwardClustered::collectLightForSlice`
(Ogre2) Assigns lights to screen-space tiles for Ogre2's forward+ rendering pipeline. Reduces per-pixel light iteration cost by pre-sorting lights into spatial clusters.

---

6 Cross-Reference Analysis

6.1 Gazebo Framework Overhead (visible in static/non-physics worlds)

Component	Where observed	% range	Optimization potential
`BaseView::ResetNewEntitiesSet`	sensors, gpu_lidar	0.5-0.9%	ECS view reset per step — investigate if needed for unchanged entities
`Barrier::Wait` + `pthread_cond_wait`	sensors	1.0-1.8%	Thread synchronization in system update scheduling
`_rmt_BeginCPUSample` (Remotery)	all worlds	1.5%	Profiler overhead; disappears with ENABLE_PROFILER=OFF

Component	Where observed	% range	Optimization potential
`SceneBroadcasterPrivate::SceneGraphAddEntities`	3k_shapes loading	3% of loading	Scene graph construction + SDF serialization scales linearly with entity count
`stbi_*` JPEG decode	jetty headless	~6%	Texture decoding during runtime — should be cached

6.2 Physics Stack (dynamic worlds)

Component	3k_shapes	jetty	Notes
`dxHashSpace::collide`	low	**21.9%**	Scales with collision pair count
`BoxedLcpConstraintSolver`	2.1%	<1%	Scales with contact count
`BodyNode::getSkeleton`	4.5%	1.1%	Per-body accessor
`BitmaskContactFilter` (Gazebo-owned)	<1%	1.5%	Per-pair filter
`OdeCollisionObject::updateEngineData` (Gazebo-owned)	<1%	1.2%	Per-object sync
Memory churn (_int_free + malloc*)	<2%	1.3%	Per-step contact allocations

6.3 Rendering Stack (sensor worlds)

Component	gpu_lidar	sensors_demo	Notes
NVIDIA EGL driver	6.9%	5.6%	GPU driver overhead
Ogre2 scene-graph traversal	~3%	**~22%**	Scales with sensor count
Ogre2 culling	2.2%	1.5%	Frustum/LOD
Ogre2 render queue	<1%	1.1%	Draw call submission

---

7 Optimization Recommendations (Priority-Ranked, Gazebo-Owned)

7.1 Priority 1: Cache texture/image decoding in jetty-class worlds

- **Evidence**: `stbi__YCbCr_to_RGB_simd` + `stbi__jpeg_decode_block` + `stbi__convert_format` at ~6% in jetty headless
- **Impact**: ~6% CPU reduction for mesh-heavy worlds
- **Call path**:

```
RenderThread → SensorsPrivate::RunOnce → RenderUtil::Update
  → SceneManager::CreateVisual → LoadGeometry → CreateMesh
    → Ogre2MeshFactory::LoadImpl → BaseMaterial::CopyFrom
      → Ogre2Material::SetTextureMapDataImpl
        → gz::common::Image::RGBAData
```

```
      → Image::Implementation::DataWithChannels
        → stbi__convert_format / stbi__YCbCr_to_RGB_simd
```

- **Root cause**: Visual creation happens **lazily during simulation** — `SceneManager::CreateVisual` is called from `RenderUtil::Update` on the render thread, not at startup. Each time a new visual is created, all its material textures are decoded from JPEG via `Ogre2Material::SetTextureMapDataImpl → gz::common::Image::RGBAData → stbi_*`. This means texture decoding competes with simulation on the render thread.
- **Location**: `gz-rendering/ogre2/src/Ogre2Material.cc` (`SetTextureMapDataImpl`), `gz-common/src/Image.cc` (`RGBAData`, `DataWithChannels`)
- **Fix options**:
  - Cache decoded `Image::RGBAData` results so the same texture isn't decoded twice across materials that share it
  - Move visual creation to a pre-simulation loading phase instead of lazy creation during `RenderUtil::Update`
  - Pre-decode textures at world load time on a background thread, so the render thread never blocks on JPEG decode

7.2 Priority 2: Reduce broadphase cost for worlds with many static bodies

- **Evidence**: `dxHashSpace::collide` at 2.6% in 3k_shapes_static (no dynamic bodies), 21.9% in jetty
- **Impact**: The broadphase dominates collision-heavy worlds. In mixed worlds (jetty: 300+ static models, 1 mobile robot), ODE sweeps all shapes including static-vs-static pairs that can never produce new contacts.
- **Call path**:

```
SimulationRunner::Step → UpdateSystems → Physics::Update
  → PhysicsPrivate::Step
    → SimulationFeatures::WorldForwardStep
      → dart::simulation::World::step              ← called unconditionally
        → ConstraintSolver::solve
          → ConstraintSolver::updateConstraints
            → GzOdeCollisionDetector::collide       ← Gazebo-owned wrapper
              → OdeCollisionDetector::collide
                → dxHashSpace::collide              ← sweeps ALL shapes
```

- **Current state**: gz-physics uses a **single collision group per world** (via `getConstraintSolver() → getCollisionGroup()`). All bodies — static and dynamic — go into the same group. `BitmaskContactFilter` filters by user-defined bitmask only and never checks whether a body is static. gz-sim *does* track static entities in `PhysicsPrivate::staticEntities` (Physics.cc:365), but this information never reaches the collision layer.
- **Location**:
  - `gz-physics/dartsim/src/EntityManagementFeatures.cc` (BitmaskContactFilter)
  - `gz-physics/dartsim/src/SimulationFeatures.cc` (WorldForwardStep)
  - `gz-sim/src/systems/physics/Physics.cc` (staticEntities tracking)
- **Fix options (incremental)**:
  1. **Quick win — filter static-vs-static in `BitmaskContactFilter::ignoresCollision`**: Add an early-return when both bodies belong to immobile skeletons (DART already exposes `Skeleton::isMobile()`). This skips narrowphase and contact generation for static-vs-static pairs but doesn't eliminate the broadphase sweep itself.
  2. **Architectural — separate collision groups**: Create two collision groups: one for dynamic bodies, one for static. Run broadphase as dynamic-vs-dynamic and dynamic-vs-static only, skipping static-vs-static entirely. This would require changes to how gz-physics constructs and manages DART collision groups, and would need to handle bodies transitioning between static and

dynamic.

3. **Full-static shortcut**: In `WorldForwardStep`, if all bodies in the world are static, skip `world->step()` entirely. Edge case but cheap to implement.

- **Note**: The broadphase sweep (`dxHashSpace::collide`) is ODE-internal and runs before any Gazebo filter is called. Options 1 and 3 help but don't touch the broadphase itself. Only option 2 eliminates the static-vs-static broadphase pairs at the ODE level.

7.3 Priority 3: Optimize `BitmaskContactFilter::ignoresCollision` and `OdeCollisionObject::updateEngineData`

- **Evidence**: Combined 2.5-2.7% in collision-heavy worlds (jetty)
- **Impact**: Small per-world, but these are called per-pair and per-object — scales with scene complexity
- **Location**: `gz-physics/dartsim/src/GzCollisionDetector.cc`, `EntityManagementFeatures.cc`
- **Fix**: Inline bitmask check; dirty-flag on updateEngineData to skip unchanged objects

7.4 Priority 4: Reduce loading time for large entity counts

- **Evidence**: 3k_shapes takes 26s to load; `SceneGraphAddEntities` visible in 3k_shapes, recursive `Element::ToString` → `GetExplicitlySetInFile` visible in jetty
- **Impact**: Critical for large-scale simulation startup
- **Location**: `gz-sim/src/systems/scene_broadcaster/SceneBroadcaster.cc`, `gz-sim/src/EntityComponentManager.cc`
- **Fix**: The bottleneck is `SdfModelSerializer::Serialize`, which converts every entity's SDF to string via recursive `Element::ToString` on first PostUpdate. Consider lazy serialization (serialize only when a subscriber requests), caching the serialized form so it isn't recomputed, or replacing the string-based serialization with a binary format. Additionally, `ServerPrivate::DownloadAssets` wastes 9% of loading time. The server parses the SDF twice by design — once for fast GUI startup (discarding models), then again on a background thread to resolve Fuel URIs and recover the models. When no Fuel URIs are present, the first parse already has everything needed. The fix: detect worlds with no Fuel URIs and keep the models from the first parse instead of discarding and re-parsing them. Finally, `pybind11::initialize_interpreter` adds a fixed ~200–260ms to every server startup by unconditionally initializing the Python interpreter (1–7.5% of loading depending on world complexity). Lazy initialization would eliminate this cost for the majority of worlds that don't use Python plugins.

7.5 Priority 5: Investigate ECS View and thread barrier overhead

- **Evidence**: `BaseView::ResetNewEntityState` (0.5-0.9%), `Barrier::Wait` (1%), `pthread_cond_wait` (1.8%) in sensor worlds
- **Impact**: Small per-world, but represents framework tax on every step
- **Location**: `gz-sim/src/detail/View.hh`, `gz-sim/src/Barrier.cc`
- **Fix**: Investigate if view state reset is needed when entities haven't changed; consider lock-free barriers

---

8 Instrumentation Gaps

- **`[libdart.so.6.13.2]` unresolved**: 1.5-10.2% across worlds. Rebuilding DART with `-g` would expose internal hotspots. Especially important for 3k_shapes_dynamic where 10.2% is opaque.
- **`[unknown]` frames**: 10-35% in several worlds — kernel stacks and stripped system libraries. `--call-graph dwarf` or installing `linux-image-dbgsym` would resolve some.
- **`void/operator/unsigned` demangling artifacts**: 12-35% — FlameGraph's stack-folding splits on `;` inside demangled C++ template signatures. Not real functions; their cost belongs to the parent frame.
- **Ogre2 internals**: Scene-graph traversal is 22% in sensors_demo but Ogre2 doesn't have debug symbols. Rebuilding Ogre2 with `-g` would reveal whether the cost is in dirty-flag checking, matrix multiplication, or memory layout.

9 Appendix

9.1 A: Flamegraph SVG Locations

Runtime (in `profiling/captures/flamegraphs/runtime/`):

```
3k_shapes_static.svg
3k_shapes_dynamic.svg
sensors_nonrendering.svg
jetty_headless.svg
gpu_lidar.svg
sensors_demo.svg
```

Loading (in `profiling/captures/flamegraphs/loading/`):

```
3k_shapes_static_loading.svg
3k_shapes_dynamic_loading.svg
sensors_nonrendering_loading.svg
jetty_loading.svg
gpu_lidar_loading.svg
sensors_demo_loading.svg
```

9.2 B: GZ_PROFILE Instrumentation Added

17 profile points added to gz-physics/dartsim: - `SimulationFeatures.cc`: 7 points (WorldForwardStep + 6 new sub-phases) - `GzCollisionDetector.cc`: 4 points (ODE + Bullet collide wrappers) - `SDFFeatures.cc`: 5 points (ConstructSdf* methods) - `KinematicsFeatures.cc`: 1 point (FrameDataRelativeToWorld)

9.3 C: Loading Wall-Clock Times

World	Time
3k_shapes_static	26.0s
3k_shapes_dynamic	26.6s
jetty	7.8s
sensors_nonrendering	2.4s
gpu_lidar	0.06s
sensors_demo	0.03s

9.4 References

[1] B. Gregg, "Flame Graphs," *ACM Queue*, vol. 14, no. 2, pp. 91-110, 2016. Software available at https://github.com/brendangregg/FlameGraph